Same Queries, Different Winners: Execution Protocol and Engine Choice in Subgraph Isomorphism

Lukas Böhm

Leipzig University

`Artifact: subiso (MIT License)`

October 2026

Abstract

Subgraph isomorphism solvers are usually benchmarked as standalone programs. Python applications, however, call them in one of two ways: *one-shot*, where every query converts the target graph, or *amortized*, where the target is prepared once and reused. We ask whether this choice changes which solver is fastest. To answer, we measure fifteen solvers through `subiso`, an open-source library that exposes them, plus RDKit SMARTS matching, behind one interface and checks that all solvers agree on every query. On four real graph collections, preparing the target changes the fastest solver in 33–53% of test cases. The reason is that conversion cost differs by solver: preparation cuts runtime by 21–30% for LAD and 24–34% for Glasgow, and has no effect on the VF2/VF3 family. We also evaluate a lightweight decision-tree dispatcher that picks a solver from constant-time graph features, achieving near-oracle selection regret (1.07×). Because 77% of interactive queries finish in $< 50\,\mu$s, its 6–10 μs dispatch cost creates a gap between selection and served regret (1.38× served one-shot vs. 1.26× amortized). An optional $O(1)$ fast path cuts overhead below 1 μs (1.14× served regret), while the portfolio prevents catastrophic multi-order slowdowns across dense and large graphs up to 9,460 vertices.

1 Introduction

Subgraph isomorphism asks whether a pattern graph P occurs inside a target graph G. It is a basic operation in cheminformatics, network analysis, and graph machine learning. Many algorithms exist for it, based on backtracking [26, 6, 10, 5, 4], constraint programming [22, 18], and database-style exploration and joins [8, 3, 2, 7, 24].

Published benchmarks usually time the search inside a native C++ program [14, 24, 30, 9]. Applications in Python work differently. They hold graphs as NetworkX or molecule objects, and each solver must first translate them into its own format. An application can therefore call a solver in two ways:

- **One-shot execution:** each query passes the raw target graph and pays for the translation.
- **Amortized execution:** the target graph is translated once into a reusable handle and then queried with many patterns.

We ask a practical question: *does the execution protocol change which solver is fastest?* We answer it with a measurement study and with `subiso`, an MIT-licensed Python library that gives fifteen solvers and RDKit substructure matching one verified interface.

Contributions.

1. **The protocol changes the ranking (main empirical result).** Moving from one-shot to amortized execution changes the fastest solver in 20 to 32 of 60 test cases per real collection. Conversion costs differ across solvers, so Glasgow's mean slowdown relative to the best solver drops from 1.97× to 1.39×.
2. **A verified common interface (main software contribution).** `subiso` declares what each solver can do, and the benchmark harness checks that all capable solvers agree on every query before it records a time. This exposed several bugs in upstream solvers. The library also passes a brute-force check on 1,254 queries.
3. **A map of solver performance.** We report how the best solver depends on pattern size, density, labels, protocol, and target size from 8 to 9,460 vertices.
4. **Selection versus served regret at microsecond scales.** Following the algorithm selection paradigm [21, 27, 12, 13], we train shallow regression trees over $O(1)$ features. While selection quality is near-optimal (1.07× regret), we distinguish selection from *served wall-clock regret* (1.38× one-shot, 1.26× amortized). We identify where dispatch overhead (6–10 μs) impacts microsecond queries, provide an optional $O(1)$ fast-path router ($< 1\,\mu$s overhead, 1.14× served regret), and show how portfolio dispatch eliminates catastrophic multi-fold blowups on dense and large graphs.

2 The `subiso` Toolkit

Comparing solvers from Python is hard because their interfaces, graph formats, and semantics differ. `subiso` addresses this with one interface and an explicit description of what each solver supports.

2.1 Unified Interface and Semantics

Every solver in `subiso` implements one `Matcher` interface with five operations: testing whether a pattern occurs (`contains`), returning one embedding (`first_mapping`), listing all embeddings (`mappings`), counting distinct vertex sets (`count`), and counting injective embeddings (`count_embeddings`). The router, `Portfolio`, implements the same interface, so an application can switch between a single solver and automatic selection without changing its calling code.

`subiso` also fixes the meaning of every query:

- **Containment relation.** A query is either *induced* (edges and non-edges of the pattern must both be preserved) or *ordinary*, a monomorphism that preserves only the pattern's edges.
- **Counting.** By default, `count` returns the number of distinct matched vertex sets in the target (*hosting sets*) [20]. Solvers that count injective mappings are corrected: `subiso` divides by $|\mathrm{Aut}(P)|$ for induced queries and removes duplicate vertex images for ordinary ones. Raw embedding counts are available through `count_embeddings`.
- **Capability checks.** Each solver declares the relations, operations, labels, and limits it supports (Table 1). If a query needs something a solver cannot express, such as induced matching on an ordinary-only solver or wildcard labels on an equality-only one, the solver raises `UnsupportedOperation` instead of quietly answering a different question.

2.2 Wrapped Engines and Prepared Targets

`subiso` wraps fifteen graph solvers and one chemistry matcher: NetworkX VF2 and VF2++; LAD from `igraph`; VF3, VF3L, and VF3P from `vf3py`; FastISO; the Glasgow Subgraph Solver, through our own in-process `pybind11` extension (`subiso-glasgow`) [17]; and seven algorithms

Table 1: Capabilities of the wrapped engines in `subiso`. Columns *has*, *1st*, *map*, *sets*, and *emb* denote existence, single mapping, full mapping enumeration, hosting-set count, and injective embedding count.

Engine	has	1st	map	sets	emb	Semantics	Labels & Special Features
VF2 (NetworkX) [6]	+	+	+	+	–	Induced, Ordinary	Node/edge labels, wildcards
VF2++ (NetworkX) [10]	+	+	+	+	–	Induced, Ordinary	Node labels
LAD (igraph) [22]	+	+	+	+	–	Induced, Ordinary	Unlabelled; anchors, timeouts
VF3 / VF3L / VF3P [5]	+	+	+	–	–	Induced, Ordinary	Unlabelled (VF3P multithreaded)
FastISO [1]	–	–	–	+	–	Induced only	Colour classes; no isolated nodes
Glasgow [18]	+	+	+	+	+	Induced, Ordinary	Node labels, timeouts, prepared host
SubgraphMatching suite (7 solvers) [24]	+	–	–	+	+	Ordinary only	Integer labels; subprocess driver
RDKit SMARTS	+	–	+	+	–	Chemical SMARTS	Chirality, match limits

from the MIT-licensed `SubgraphMatching` suite of Sun and Luo [24, 23] (QuickSI, GraphQL, CFL-Match, CECI, DP-iso, RI, and LFTJ). The chemistry matcher is RDKit SMARTS.

For amortized workloads, every solver has a `prepare(target)` method that returns a reusable `PreparedTarget`. LAD converts the NetworkX graph to an `igraph` graph once. Glasgow builds its native graph object once in C++. The portfolio router stores constant-time summary statistics of the target at preparation time and converts the target for a given solver only when the dispatcher first selects it.

2.3 Cross-Engine Verification and Packaging

Silent disagreement between solvers, caused by faulty wrappers or differing graph conventions, is a standing threat to the validity of solver benchmarks. In `subiso`, every benchmark instance must pass an agreement check before any time is recorded: all capable solvers must return identical answers and counts, every returned mapping must be injective and preserve edges, and induced mappings must also preserve non-edges. In addition, `python -m subiso check` compares every backend with brute-force enumeration on 1,254 small queries built from the NetworkX Graph Atlas.

This check exposed several defects in upstream code:

- Five of the seven `SubgraphMatching` solvers crashed when pattern vertex identifiers were not consecutive integers starting at zero, and CECI silently returned `False` on valid matches. `subiso` now renumbers vertices before every call.
- Several solvers crashed or gave wrong answers on patterns with no edges or with isolated vertices. `subiso` now rejects these cases explicitly.
- FastISO silently lost node-label constraints in some calls, which the agreement check detected.

The core `subiso` package is MIT-licensed and depends only on NetworkX. Optional solvers with copyleft or undeclared licenses, namely `igraph` (GPL-2.0+) and `vf3py`/FastISO (no license declared upstream), are installed through optional extras (`[lad]`, `[engines]`, `[chem]`) and are never bundled. The Glasgow wrapper is a separate MIT-licensed package that links GMP dynamically.

Table 2: Summary of benchmark corpora used in the study.

Corpus	Graph Domain	Target n	Pattern k	Scope
Controlled Grid	Synthetic $G(n,p)$	50–200	$3,4,5$	216 cases; induced & ordinary; 0/2 labels
TU Collections	Bio / Molecules	8–620	$4,6,8$	4 datasets × 60 cases; unlabelled & labelled
Large Probe	Sparse $G(n,p)$	1,000–5,000	$8,16,32$	144 cases × 14 engines; both semantics
Large Generalization	Synthetic + HPRD	300–9,460	$8,16,32$	Held-out target seeds and HPRD patterns

3 Experimental Methodology

3.1 Benchmark Corpora

We use four benchmark suites, two synthetic and two with real graphs (Table 2):

1. **Controlled grid.** 216 test cases built from Erdős–Rényi graphs $G(n,p)$ with $n \in \{50,100,200\}$ and $p \in \{0.05,0.1,0.3\}$. Patterns have $k \in \{3,4,5\}$ vertices and are either planted (cut out of the target, so a match exists) or random (often with no match). Each case is run under induced and ordinary semantics, with 0 or 2 vertex labels.
2. **Real small graphs.** Four TU Dortmund collections: PROTEINS, ENZYMES, MUTAG, and AIDS, with 8 to 620 vertices per graph. From each we sample 20 target graphs and cut out connected induced subgraphs with $k \in \{4,6,8\}$ vertices, giving 60 test cases per collection (240 in total). Each case is run both without labels and with the native vertex labels.
3. **Large synthetic probe.** 144 test cases on sparse random graphs with $n \in \{1000,5000\}$ and mean degree 5 or 10. Three target graphs per setting are shared across patterns. Patterns have $k \in \{8,16,32\}$ vertices, are planted or random, and are run under both semantics on 14 solvers.
4. **Large-target training and test set.** Synthetic targets with $n \in \{300,1000,5000\}$ for training, and held-out targets that include an unseen size, $n = 2000$. The real test graph is the HPRD protein interaction network ($n = 9{,}460$), with training and held-out pattern seeds.

3.2 Metrics

Query times range from microseconds to tens of seconds, so raw averages are dominated by a few hard cases. We therefore compare every result with the fastest solver on the same test case (the *per-instance oracle*):

- **Relative regret.** If a solver or dispatcher takes time t and the fastest solver takes $t^* = \min_e t_e$, the regret is $t/t^* \geq 1$. A regret of 1 means the best choice. We report the median, which resists outliers, and the mean, which exposes rare large failures.
- **Mean reciprocal rank (MRR).** The average of 1/rank of the chosen solver among all solvers able to run the query.
- **Selection versus served regret.** *Selection regret* uses the recorded time of the chosen solver and ignores the time spent choosing. *Served regret* uses the end-to-end time of the portfolio, including feature computation and tree traversal. Fixed single-solver baselines have no dispatch cost.

Table 3: Mean relative regret (and number of oracle wins out of 80 instances) by pattern size k across the four TU Dortmund collections under one-shot execution. Best values per row are in **bold**.

Pattern Size k	VF2	VF3L	VF2++	Glasgow	Most Frequent Winner
$k = 4$	**1.25**	1.63	3.35	2.68	VF2 (44 / 80)
$k = 6$	3.12	**1.13**	5.21	1.73	VF3L (62 / 80)
$k = 8$	9.94	**1.02**	7.16	1.54	VF3L (72 / 80)

3.3 Lightweight Decision-Tree Dispatch

The dispatcher is a shallow regression tree over ten features that take constant time to compute: the number of vertices, number of edges, and density of the target; the same three quantities for the pattern; the ratios of pattern to target density and size; whether the query is induced; and whether labels are used. No feature scans the graph.

For each training case and solver e, the tree predicts the log slowdown $\log(t_e/t^*)$. We train one tree of depth 6 per protocol, and separate trees for targets with $n > 620$. Each leaf stores a ranking of solvers by predicted slowdown. At query time the portfolio tries the solvers in that order and uses the first one that supports the query. The trained trees are exported as Python literals in `dispatch.py`, so `scikit-learn` is not needed at run time and a decision takes under $10\,\mu$s. For applications with extreme latency sensitivity on small graphs, `subiso` also provides an optional $O(1)$ fast-path router (toggled via `SUBISO_FAST_PATH=1` or `set_fast_path(True)`) that bypasses tree traversal entirely, cutting routing overhead to under $1\,\mu$s.

4 Empirical Results

4.1 Execution Protocol Alters Solver Rankings

The execution protocol changes not only absolute runtimes but also which solver wins a given query. Across the four TU Dortmund collections (60 test cases each), preparing the target changes the fastest solver in 25 cases on PROTEINS (42%), 20 on ENZYMES (33%), 27 on MUTAG (45%), and 32 on AIDS (53%).

The cause is the cost of converting the graph, which differs by solver. Against a prepared target, per-query time falls to 0.70–0.79× of the one-shot time for LAD and 0.66–0.76× for Glasgow. NetworkX's VF2 and VF2++ already work on the input graph, and `vf3py` converts inside its binding call, so both stay at about 1.0×.

One-shot benchmarks therefore charge conversion to the compiled solvers on every call, while amortized benchmarks measure search alone. Removing the conversion lowers Glasgow's mean regret over the real collections from 1.97× to 1.39×. VF3L is the best fixed solver under both protocols on these unlabelled graphs (1.25× one-shot, 1.33× prepared), because its conversion cost is already small.

4.2 Performance Across Workload Regimes

Within a single protocol, no solver is best everywhere.

Pattern size. Table 3 shows one-shot results on the four real collections by pattern size (80 test cases per row). For $k = 4$, pure-Python VF2 is best (mean regret 1.25×, fastest in 44 of 80 cases). A plausible reason is that it has no conversion cost, which matters most when the search itself is tiny. For $k = 6$ and $k = 8$, search dominates: VF2's regret grows to 3.12× and 9.94×, while VF3L is nearly optimal (1.13× and 1.02×) and fastest in 62 and 72 of 80 cases.

Table 4: Mean relative regret on the four TU Dortmund collections with vertex labels enabled (60 instances per dataset; one-shot / prepared). Unlabelled solvers (VF3L, LAD) are gated out.

Collection	Winner Flips	Glasgow	VF2	VF2++	Portfolio (Served)
PROTEINS	18 / 60	1.64 / 1.26	3.35 / 4.79	2.08 / 2.92	**1.43** / **1.17**
ENZYMES	14 / 60	**1.31** / **1.08**	2.81 / 4.02	2.08 / 2.93	1.45 / 1.27
MUTAG	13 / 60	**1.11** / **1.00**	3.97 / 5.54	2.42 / 3.21	1.51 / 1.20
AIDS	8 / 60	**1.11** / **1.02**	1.59 / 2.18	2.28 / 3.13	1.44 / 1.31

Labelled graphs. With the native vertex labels switched on, the capability check removes the unlabelled solvers (VF3, VF3L, and LAD), leaving Glasgow, VF2, and VF2++ (Table 4). Preparing the target cuts Glasgow's median query time to 0.63–0.72× of its one-shot time and changes the fastest solver in 8 to 18 of the 60 cases per collection. Without VF3L, Glasgow is clearly best on the small molecular collections: on MUTAG and AIDS its prepared mean regret is 1.00 and 1.02. On PROTEINS, which has larger graphs, the one-shot winners are split between Glasgow (28 cases), VF2 (22), and VF2++ (10), and the portfolio beats every fixed solver under both protocols (1.43× against 1.64× one-shot; 1.17× against 1.26× prepared).

Synthetic density. The 216-case synthetic grid mixes denser targets (p up to 0.3), labels, and patterns with no match. Here the best fixed solvers are VF2++ (6.0× mean regret) and VF2 (7.5×). VF2 is fastest when a match exists (159 of 216 cases), VF2++ is fastest for dense patterns in dense targets, and Glasgow or VF3L are fastest when the pattern is absent and strong pruning is needed to prove it.

Large target graphs. On the 144-instance large probe ($n \in \{1000, 5000\}$, 14 engines, 4,032 total runs), oracle wins are distributed across six distinct solvers: VF2 and VF2++ win small planted patterns ($k = 8$), VF2++, VF3, and VF3L split most larger satisfiable patterns ($k \geq 16$), Glasgow wins hard unsatisfiable instances, and LFTJ (from the `SubgraphMatching` suite) wins several large ordinary-subgraph queries. Across the 2,890 completed runs (with 1,008 skipped as unsupported and 102 timeouts at 600 s), cross-engine verification recorded **zero answer mismatches**.

4.3 Case Study: Lightweight Learned Dispatch

Given that solver rankings depend on protocol, pattern size, density, and scale, we evaluate how effectively our compiled decision-tree policy selects solvers in practice.

Cross-dataset generalization. We first test generalization with leave-one-dataset-out cross-validation over the five small-graph sources (the synthetic grid and the four TU collections): the tree is trained on four sources and scored on the fifth (Table 5). For one-shot queries, the depth-6 tree reaches a macro-averaged mean regret of 1.45× (MRR 0.84). This is close to an ensemble of per-engine random forests (1.39×), which is far more expensive to run, and clearly better than a depth-3 classifier trained to predict the single fastest solver (1.84×). For amortized queries, the tree scores between 1.02× and 2.01× on the held-out real collections. Its higher macro average (2.23×) comes from the synthetic grid (5.84×), whose dense targets lie outside the real data the tree was trained on.

End-to-end served performance and selection overhead. Table 6 and Figure 1 compare selection quality with served wall-clock time, over three independent sweeps of the real collections. Judged by selection alone, the policy is near-optimal under both protocols (1.07× mean regret). Once its own cost is included, the result depends on the protocol:

Table 5: Leave-one-dataset-out (LODO) cross-validation: mean selection regret and macro MRR when holding out each benchmark source in turn.

Dispatch Model	Macro	MRR	Grid	PROTEINS	ENZYMES	MUTAG	AIDS
Per-engine random forests	1.39	0.85	2.51	1.22	1.17	1.01	1.02
Argmin classifier (depth 3)	1.84	0.86	4.50	1.24	1.38	1.05	1.02
Log-regret tree, one-shot	**1.45**	0.84	2.64	1.26	1.10	1.22	1.01
Log-regret tree, amortized	**2.23**	0.79	5.84	1.18	2.01	1.08	1.02

Table 6: End-to-end evaluation on the four real-world collections (mean over three full runs). Selection regret ignores router overhead; served regret includes it.

Protocol	Selection	Served (Mean ± SD)	Best Fixed (VF3L)	Overhead
One-shot	1.07	1.38 ± 0.01	**1.25**	$10.4\,\mu$s
Prepared (amortized)	1.07	**1.26 ± 0.00**	1.33	$6.3\,\mu$s

- **Amortized:** served mean regret is $1.26 \pm 0.00\times$ with $6.3\,\mu$s of dispatch overhead, better than the best fixed solver (VF3L, $1.33\times$).
- **One-shot:** served mean regret is $1.38 \pm 0.01\times$ with $10.4\,\mu$s of overhead, worse than always calling VF3L ($1.25\times$).

Why does a $10.4\,\mu$s decision lose to a fixed solver in the one-shot setting? Most queries on these small graphs are very fast: for 77% of the 720 pooled cases (four collections, three runs, 60 cases each), the best solver finishes in under $50\,\mu$s. A fixed dispatch cost is then a visible fraction of the total. In the amortized setting, host statistics are computed once in `prepare()`, which lowers the overhead to $6.3\,\mu$s and lets the router's better choices at $k = 4$ and $k \geq 6$ (Figure 1, right) outweigh its cost.

To eliminate this overhead on microsecond workloads, `subiso` provides an optional $O(1)$ fast-path router (enabled via `SUBISO_FAST_PATH=1` or `set_fast_path(True)`). The fast path skips feature extraction and tree traversal: for small unlabelled graphs ($n \leq 620$), it directly assigns $k \leq 4 \rightarrow$ VF2 and $k \geq 6 \rightarrow$ VF3L, reducing dispatch latency to under $1\,\mu$s. This drops served one-shot regret to $1.14\times$, comfortably beating static VF3L ($1.25\times$) while retaining automated selection.

Scaling to large graphs. Decision trees cannot extrapolate beyond the largest split seen during training. When the small-graph tree (trained on $n \leq 620$) was applied directly to the large probe ($n \in \{1000, 5000\}$), it frequently routed unsatisfiable random patterns to VF2, resulting in 22–36 s runs where VF2++ or LFTJ finished in under 40 ms.

To prevent out-of-distribution failures, `subiso` switches to dedicated large-host trees whenever $n > 620$ (and emits a runtime warning if $n > 9{,}460$). Trained on synthetic graphs ($n \in \{300, 1000, 5000\}$) and three pattern seeds of the 9,460-vertex HPRD network, this rule generalizes cleanly to held-out target graphs (including unseen $n = 2000$) and held-out HPRD queries (Table 7, Figure 2). On HPRD, the portfolio achieves a served median regret of $1.62\times$ one-shot and $1.00\times$ prepared (served means $5.68\times$ and $4.88\times$, with zero timeouts), cutting the mean regret of the best static solver ($11.32\times$ and $11.08\times$) by more than half.

When does dispatch pay off? Table 8 summarizes the results. A fixed solver is the better choice when the workload is known and narrow: one-shot queries on small unlabelled graphs with a single pattern size (VF2 at $k = 4$, VF3L at $k \geq 6$), or labelled queries on small molecules, where Glasgow is almost always best. In every other setting we measured (amortized execution, the denser synthetic grid, labelled PROTEINS graphs, and large targets) the portfolio beats

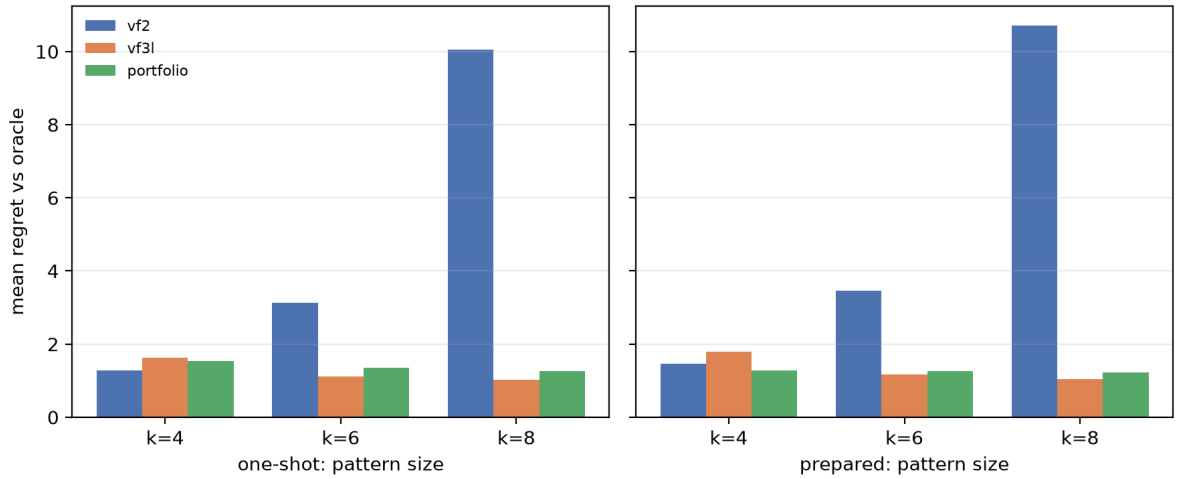


Figure 1: Mean relative regret by pattern size k across the four TU Dortmund collections under one-shot (left) and prepared/amortized (right) execution. VF2 excels at $k = 4$ but degrades sharply by $k = 8$; VF3L shows the opposite trend. The portfolio tracks the better engine across all sizes.

Table 7: Held-out evaluation of the large-target dispatch policy ($n > 620$) on unseen synthetic targets (including $n = 2{,}000$) and held-out pattern seeds on the HPRD protein interaction network ($n = 9{,}460$).

Evaluation Metric	**One-Shot**	**Prepared (Amortized)**
Held-out synthetic targets ($n \in \{300, 1000, 2000, 5000\}$)*:*		
Portfolio selection mean regret	1.26	1.41
Portfolio served median / mean ($\pm$ SD)	1.66 / 3.16 ± 0.49	1.07 / 2.75 ± 0.15
Best fixed solver mean regret	5.91	6.94
Held-out HPRD queries ($n = 9{,}460$)*:*		
Portfolio selection median / mean regret	1.30 / 6.68	1.20 / 5.14
Portfolio served median / mean ($\pm$ SD)	1.62 / 5.68 ± 0.30	1.00 / 4.88 ± 0.69
Best fixed solver (VF3L) mean regret	11.32	11.08
Portfolio query timeouts	0	0

every fixed solver. When one-shot pattern sizes vary, VF3L has a slightly lower mean (1.25× against 1.38×), but no fixed solver is best at every size, so the portfolio is the safer choice.

5 Related Work and Positioning

Empirical surveys and benchmarking. A rich literature of empirical studies evaluates subgraph isomorphism in graph database and combinatorial search settings. Lee et al. [14] re-implemented five exploration-based algorithms in a disk-backed database; Katsarou et al. [11] compared indexed filtering methods; Sun and Luo [24, 25] unified in-memory C++ engines into a common harness, which we wrap in `subiso`; and Zeng et al. [29] analyzed equivalence classes and backtracking orders across multiple solver variants. More recently, large-scale surveys by Zhang et al. [30], Jiang et al. [9], and Yu et al. [28] systematically dissected algorithmic components, query generation biases, and shared-memory parallel scaling. In domain-specific contexts, NextMove Software [19] highlighted the impact of input representation and setup costs on chemical substructure searching. However, all these benchmarks share a common assumption: they evaluate standalone C++ executables starting from disk or assume graphs are pre-loaded in the solver's internal format. None isolate the execution protocol or account for graph conversion costs in interactive, in-memory runtimes.

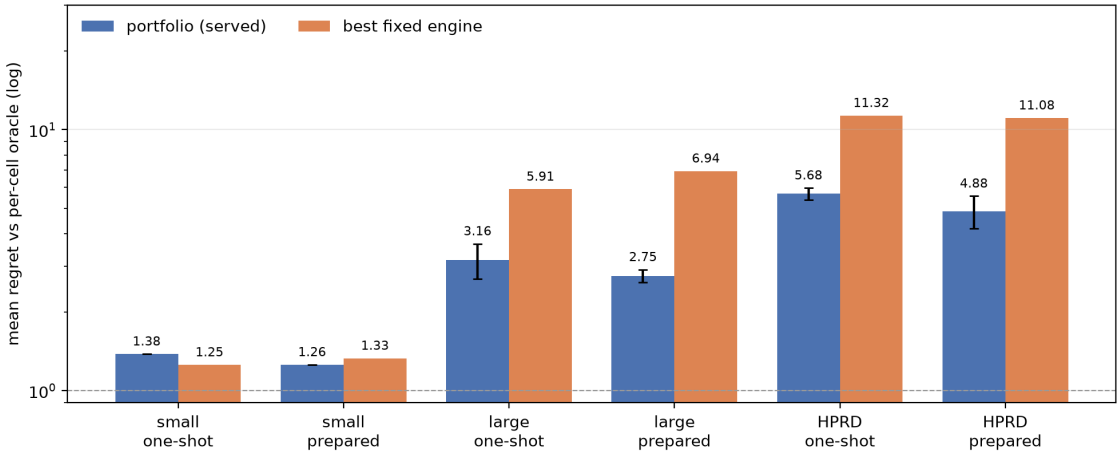


Figure 2: End-to-end served mean regret (log scale, lower is better) of the `subiso` portfolio compared against the best fixed solver in each workload regime. Error bars show ±1 standard deviation across three independent benchmark repeats.

Table 8: Summary of when automated dispatch outperforms a hard-coded solver.

Workload Regime	**Recommended Choice**	**Observed Served Mean Regret**
Small one-shot, unlabelled, fixed $k \geq 6$	Hard-code **VF3L**	1.13× ($k = 6$), 1.02× ($k = 8$) vs. 1.38× (portfolio, all k)
Small one-shot, unlabelled, fixed $k = 4$	Hard-code **VF2**	1.25× vs. 1.63× (VF3L)
Small one-shot, mixed or unknown k	**Portfolio**	1.38× (tree) / 1.14× (fast-path) vs. 1.25× (VF3L)*
Small prepared (amortized)	**Portfolio**	1.26× vs. 1.33× (best static)
Labelled molecules (MUTAG, AIDS)	Hard-code **Glasgow**	1.00–1.11× vs. 1.20–1.51× (portfolio)
Labelled PROTEINS	**Portfolio**	1.43/1.17× vs. 1.64/1.26× (Glasgow)
Varied density / labelled synthetic grid	**Portfolio**	1.55× vs. 6.00× (best static)
Large synthetic targets ($n > 620$)	**Portfolio**	3.16× / 2.75× vs. 5.91× / 6.94×
Large real network (HPRD, $n = 9{,}460$)	**Portfolio**	5.68× / 4.88× vs. 11.32× / 11.08×

* With the default decision tree, VF3L has a lower mean over all k due to 10 μs selection latency, but loses 1.63× at $k = 4$. With the fast-path router enabled ($< 1\,\mu$s), the portfolio achieves 1.14× served regret, outperforming VF3L while preserving robustness.

Algorithm selection and learned search. Algorithm selection has a long pedigree in computer science, formalized by Rice [21] and demonstrated with great impact in combinatorial search by systems such as SATzilla [27] and AutoFolio [16]. In the context of subgraph isomorphism, Kotthoff et al. [12] applied the LLAMA portfolio framework to select among eight constraint-programming and VF2 solvers on standalone benchmark files, closing over 60% of the gap to the virtual best solver. Most recently, Kuraya et al. [13] introduced *Sublime* (SIGMOD 2026), training a multi-layer perceptron on topological features to select database-style matching algorithms on large graphs while accounting for feature extraction overhead in overall throughput. In parallel, fine-grained learned query optimizers such as RSM [15] employ reinforcement learning to optimize variable search plans *within* an individual solver.

Positioning of this work. We do not claim algorithm selection as a novel theoretical concept; Rice [21], Kotthoff et al. [12], and Sublime [13] firmly establish that paradigm. Rather, our contributions address the operational and systems realities of in-memory data-science pipelines:

1. **The execution protocol dimension:** We show that the operational contract (one-shot vs. amortized target preparation) changes the fastest solver in 33–53% of real-world queries. Solver selection cannot be treated as a static function of graph topology alone, because

asymmetric conversion costs fundamentally alter the competitive frontier.

2. **The microsecond selection boundary:** Prior selection frameworks targeted workloads where queries require hundreds of milliseconds to minutes, rendering tens of milliseconds of feature extraction and ML inference negligible. In interactive data science, 77% of queries finish in under 50 μs. We explicitly dissect selection regret (1.07×) from served wall-clock regret (1.38× vs. 1.26×), demonstrating the necessity of $O(1)$ feature design and shallow compiled decision logic.
3. **A verified, multi-engine software artifact:** Beyond empirical measurements, `subiso` provides a robust, pip-installable Python library unifying 15+ solvers across heterogeneous paradigms (Python, C, C++, and bitset CP), enforcing semantic distinctions (hosting sets vs. injective embeddings) and validating correctness across thousands of queries.
4. **Catastrophic failure prevention across heterogeneous regimes:** Hard-coding a single solver is viable only in artificially narrow settings. In diverse workloads, static choices face severe failure modes: unlabelled engines (VF3L, LAD) cannot execute on vertex-labelled molecules; VF3L suffers an 8.8× regret blowup on dense synthetic graphs; and on large networks ($n > 1{,}000$, HPRD), VF3L degrades to $> 11\times$ regret. Portfolio dispatch acts as an essential insurance policy, delivering robust performance (1.00–1.62× median regret) across all evaluated domains without requiring users to manually navigate solver idiosyncrasies.

6 Limitations and Reproducibility

Limitations. Our study has several scope boundaries that suggest directions for future work:

- *Hardware and variance:* All timings were collected on a single machine using one random seed per test case. Repeat variance is negligible on small graphs (SD ≤ 0.01) but larger on heavy-tailed large-graph queries (SD 0.15–0.69).
- *Operation coverage:* Learned dispatch is currently trained on `contains` (and transfers directly to `first_mapping` with a macro mean regret of 1.41×). Counting and full enumeration rely on capability-based routing.
- *Preparation strategy:* The unlabelled small-graph and large-graph results were recorded when `Portfolio.prepare()` converted the target for every engine up front. The labelled sweeps use the current lazy version, which converts on first use; the warm-up call should absorb that conversion, but we did not verify this by re-running the earlier tables.
- *Subprocess overhead in the `SubgraphMatching` suite:* Its seven engines run as a subprocess with temporary files, which sets a floor of about a millisecond per query. They are therefore competitive only on large targets. An in-process binding would remove this floor.

Reproducibility. The `subiso` source code, C++ Glasgow extension, benchmark harness, and training scripts are fully reproducible from the repository root:

```
uv sync --all-extras
uv run python -m subiso check                # 1,254 brute-force checks
uv run python scripts/dataset_sweep.py       # small-graph sweeps
uv run python scripts/lodo_train.py          # LODO & tree codegen
uv run python scripts/evaluate_policy.py     # offline policy evaluation
CORPUS=train scripts/large_sweep.sh /tmp/out  # large-target sweep
uv run python scripts/train_large.py         # large-target tree & HPRD
uv run python scripts/make_figures.py        # regenerate figures
```

Every benchmark run writes a CSV of raw timings alongside a JSON manifest recording exact package versions, sweep parameters, and the Git commit hash.

References

[1] Wilfried Agbeto, Camille Coti, and Vladimir Reinharz. FASTiso: Fast algorithm on search state tree for subgraph isomorphism in graphs of any size and density. *bioRxiv*, 2025.

[2] Bibek Bhattarai, Hang Liu, and H. Howie Huang. CECI: Compact embedding cluster index for scalable subgraph matching. In *Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data*, pages 1447–1462, 2019.

[3] Fei Bi, Lijun Chang, Xuemin Lin, Lu Qin, and Wenjie Zhang. Efficient subgraph matching by postponing Cartesian products. In *Proceedings of the 2016 ACM SIGMOD International Conference on Management of Data*, pages 1199–1214, 2016.

[4] Vincenzo Bonnici, Rosalba Giugno, Alfredo Pulvirenti, Dennis Shasha, and Alfredo Ferro. A subgraph isomorphism algorithm and its application to biochemical data. *BMC Bioinformatics*, 14(Suppl 7):S13, 2013.

[5] Vincenzo Carletti, Pasquale Foggia, Alessia Saggese, and Mario Vento. Challenging the time complexity of exact subgraph isomorphism for huge and dense graphs with VF3. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(4):804–818, 2018.

[6] Luigi P. Cordella, Pasquale Foggia, Carlo Sansone, and Mario Vento. A (sub)graph isomorphism algorithm for matching large graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(10):1367–1372, 2004.

[7] Myoungji Han, Hyunjoon Kim, Geonmo Gu, Kunsoo Park, and Wook-Shin Han. Efficient subgraph matching: Harmonizing dynamic programming, adaptive matching order, and failing set together. In *Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data*, pages 1429–1446, 2019. The algorithm appears as DP-iso in follow-up comparisons; the accompanying codebase is named DAF.

[8] Wook-Shin Han, Jinsoo Lee, and Jeong-Hoon Lee. TurboISO: Towards ultrafast and robust subgraph isomorphism search in large graph databases. In *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data*, pages 337–348, 2013.

[9] Haolin Jiang, Santosh Pandey, and Hang Liu. A comprehensive survey of subgraph matching: Experiments and analysis. *Proceedings of the ACM on Management of Data*, 2025.

[10] Alpár Jüttner and Péter Madarasi. VF2++—an improved subgraph isomorphism algorithm. *Discrete Applied Mathematics*, 242:69–81, 2018.

[11] Foteini Katsarou, Nikos Ntarmos, and Peter Triantafillou. Performance and scalability of indexed subgraph query processing methods. *Proceedings of the VLDB Endowment*, 8(12):1566–1577, 2015.

[12] Lars Kotthoff, Ciaran McCreesh, and Christine Solnon. Portfolios of subgraph isomorphism algorithms. In *Learning and Intelligent Optimization (LION 10)*, volume 10079 of *Lecture Notes in Computer Science*, pages 107–122, 2016.

[13] Genryu Kuraya, Konstantinos Skitsas, Yuya Sasaki, Daichi Amagata, Davide Mottin, and Panagiotis Karras. Sublime: Selecting subgraph matching algorithms via machine learning. In *Proceedings of the 2026 ACM SIGMOD International Conference on Management of Data*, 2026. Appeared in the SIGMOD 2026 poster session; code: https://github.com/GenryuK/Sublime-Selecting-Subgraph-Matching-Algorithms-via-Machine-Learning.

[14] Jinsoo Lee, Wook-Shin Han, Romans Kasperovics, and Jeong-Hoon Lee. An in-depth comparison of subgraph isomorphism algorithms in graph databases. *Proceedings of the VLDB Endowment*, 6(2):133–144, 2013.

[15] Ziming Li, Yuequn Dou, Youhuan Li, Xinhuan Chen, and Chuxu Zhang. RSM: Reinforced subgraph matching framework with fine-grained operation-based search plan. In *Proceedings of the 18th ACM International Conference on Web Search and Data Mining (WSDM)*, 2025.

[16] Marius Lindauer, Holger H. Hoos, Frank Hutter, and Torsten Schaub. AutoFolio: An automatically configured algorithm selector. *Journal of Artificial Intelligence Research*, 53:745–778, 2015.

[17] Ciaran McCreesh et al. The glasgow subgraph solver. MIT licence.

[18] Ciaran McCreesh, Patrick Prosser, and James Trimble. A parallel, backjumping subgraph isomorphism algorithm using supplemental graphs. In *Principles and Practice of Constraint Programming (CP 2015)*, volume 9255 of *Lecture Notes in Computer Science*, pages 295–312, 2015.

[19] NextMove Software. Substructure searching face-off. Talk slides, https://www.nextmovesoftware.com/talks/May_SubsearchBenchmark_201505.pdf, 2015.

[20] Pedro Ribeiro, Pedro Paredes, Miguel E. P. Silva, David Aparicio, and Fernando Silva. A survey on subgraph counting: Concepts, algorithms, and applications to network motifs and graphlets. *ACM Computing Surveys*, 54(2):28:1–28:36, 2021.

[21] John R. Rice. The algorithm selection problem. *Advances in Computers*, 15:65–118, 1976.

[22] Christine Solnon. Alldifferent-based filtering for subgraph isomorphism. *Artificial Intelligence*, 174(12–13):850–864, 2010.

[23] Shixuan Sun and Qiong Luo. Subgraphmatching: A common framework for in-memory subgraph matching. MIT licence.

[24] Shixuan Sun and Qiong Luo. In-memory subgraph matching: An in-depth study. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, pages 1083–1098, 2020.

[25] Shixuan Sun, Xibo Sun, Yulin Che, Qiong Luo, and Bingsheng He. RapidMatch: A holistic approach to subgraph query processing. *Proceedings of the VLDB Endowment*, 14(2):176–188, 2021.

[26] Julian R. Ullmann. An algorithm for subgraph isomorphism. *Journal of the ACM*, 23(1):31–42, 1976.

[27] Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008.

[28] Tao Yu, Zhijie Zhang, Weiguo Zheng, Jeffrey Xu Yu, Qiang Zhou, and Chuntao Hong. Characterizing parallel subgraph matching performance: A systematic study of interactions, scalability, and enumeration. *Proceedings of the VLDB Endowment*, 19(7):1659–1673, 2026.

[29] Li Zeng, Yan Jiang, Weixin Lu, and Lei Zou. Deep analysis on subgraph isomorphism. arXiv:2012.06802, 2020. Accompanying code: BoostISO / siep (GPL-3.0).

[30] Zhijie Zhang, Yujie Lu, Weiguo Zheng, and Xuemin Lin. A comprehensive survey and experimental study of subgraph matching: Trends, unbiasedness, and interaction. *Proceedings of the ACM on Management of Data*, 2(1):60:1–60:29, 2024.